Process and Team

เรียบเรียงโดย Wiwat Vatanawood

Agenda

1. Data Science and SE Process Model
2. Interdisciplinary Teams
3. Technical Debt

1. Data Science and SE Process Model

Data Science and Software Engineering Process Models

- **Overview:** Building ML-enabled products requires the integration of two distinct worlds: Data Science and Software Engineering
- **Interdisciplinary Teams:** Success depends on the collaboration of data scientists, software engineers, product managers, and domain experts
- **The Core Challenge:** Coordinating significantly different development processes to build a single, integrated software product

เนื้อหาบทนี้ เน้นเรื่อง Integrated Process Model ระหว่าง Data Scientist และ Software Engineer

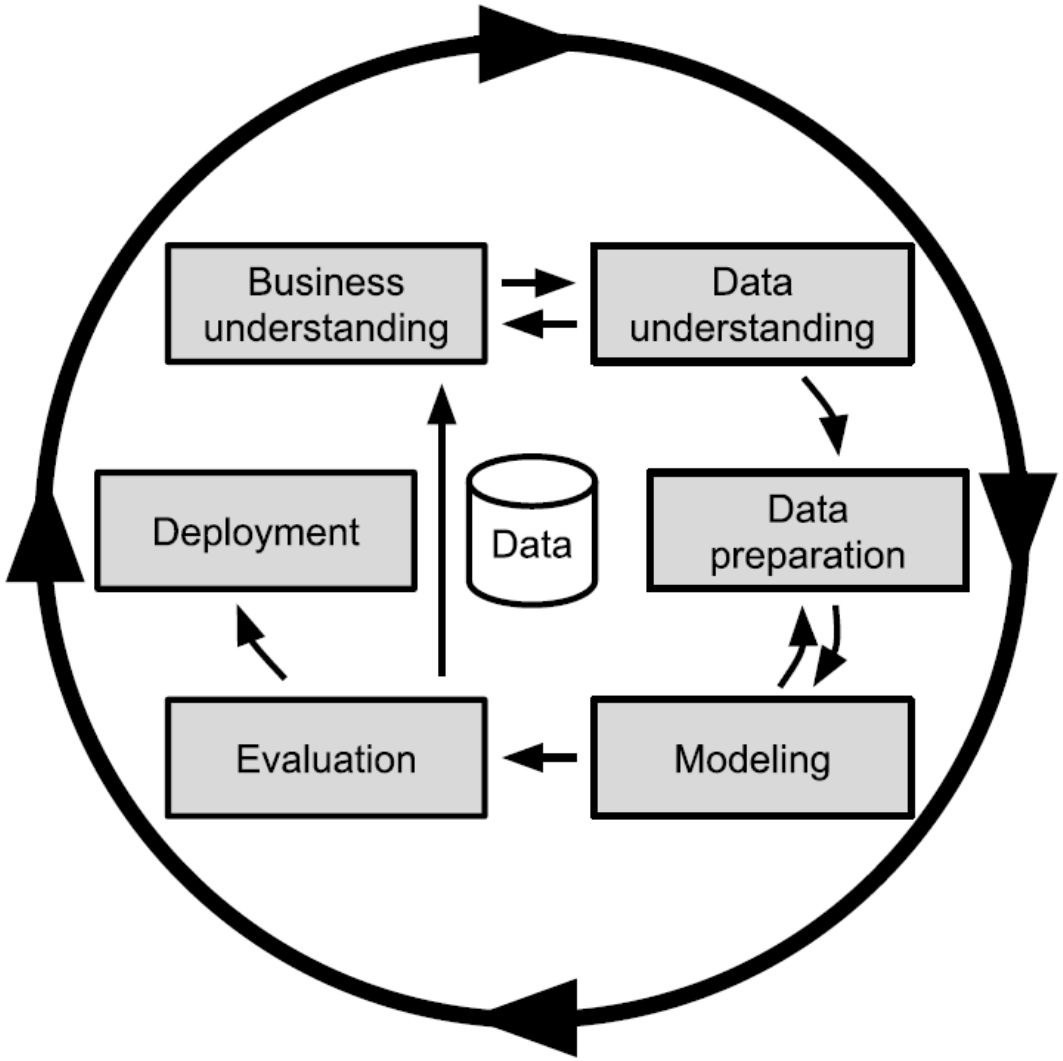
The Data-Science Process: A Science Mindset

- **Iterative and Exploratory:** Model development is rarely linear; it involves constant cycles of understanding the problem, cleaning data, and engineering features
- **The "Science" Approach:** Starts with a goal and explores whether a solution is even possible through experiments and observations
- **Unpredictability:** Unlike traditional coding, it is difficult to plan how long it will take to reach a specific accuracy goal because it is unknown which solutions will work
- **Standard Frameworks:** Common models include **CRISP-DM** (Cross-Industry Standard Process for Data Mining) and **Microsoft's Team Data Science Process**

อ้างถึง Data Science process ที่มีลักษณะ Iterative and Exploratory วนซ้ำและสำรวจไปทั่วหาคำตอบ โดยใช้หลักการ "Science" มีสมมุติฐานและทดลองหาผล ประเมิน หาคำตอบที่ดีที่สุด

CRISP-DM model by IBM

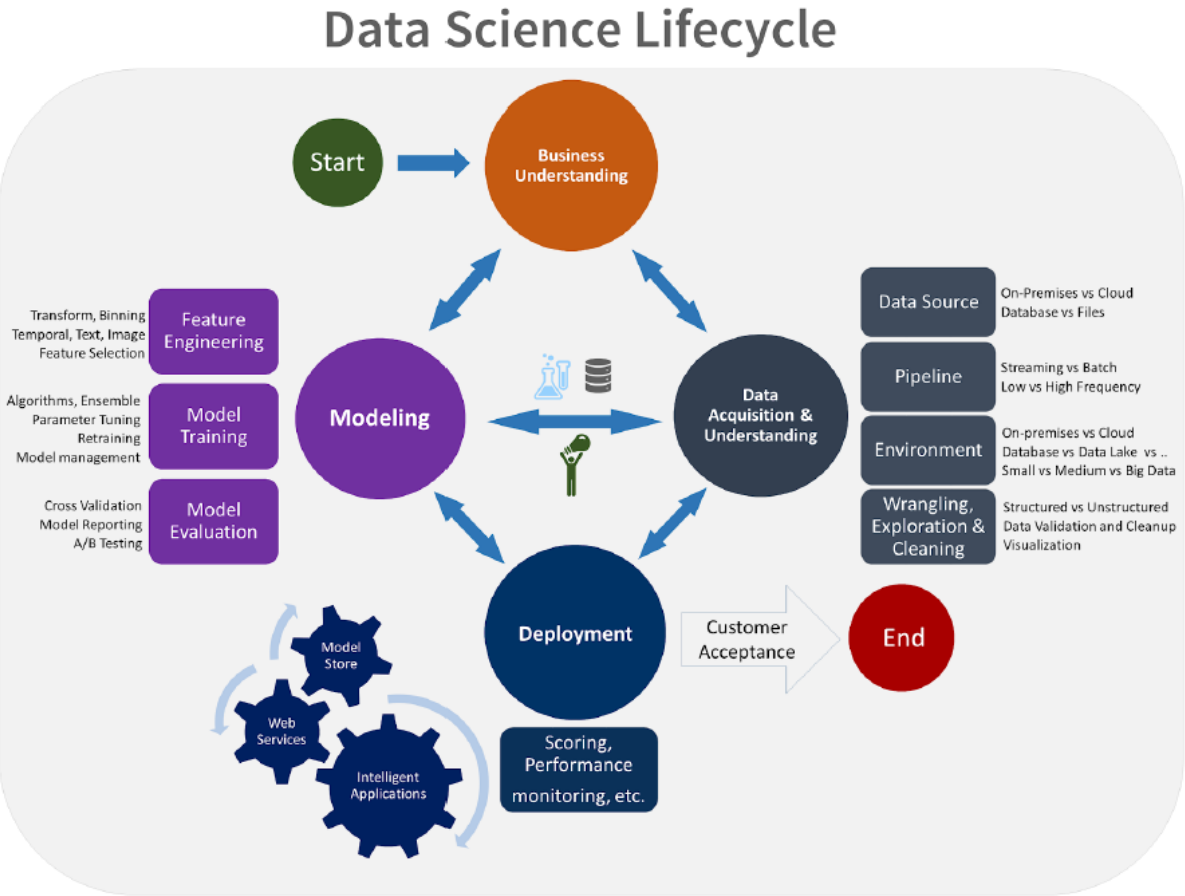
- Iterative and exploratory
- Focus on Data and Training



A process diagram showing the relationship between the different phases of CRISP-DM, adopted from IBM SPSS Modeler Documentation "CRISP-DM Help Overview" at https://www.ibm.com/docs/en/spss-modeler/saas?topic=dm-crisp-help-overview.

MS's Team Data Science Process

- Iterative and exploratory
- Focus on Data and Training



A process model of Microsoft's Team Data Science Process, from Microsoft Azure Team, "What is the Team Data Science Process?" Microsoft Documentation, Jan 2020 [Online-only figure.]

Tooling for Exploration: Computational Notebooks

- **The Role of Jupyter:** Notebooks are the standard for exploratory data analysis
- **Benefits for Exploration:**
  - **Quick Feedback:** Allows for executing short code snippets incrementally
  - **Incremental Computation:** Users can change and re-execute individual cells without running the entire program
  - **Integrated Visualization:** Shows figures and tables directly alongside the code
- **Transitioning to Production:** While great for prototyping, notebook code is often "messy" and should be migrated to robust, modular pipelines for production use

Notebooks เป็น Tool สำคัญในปัจจุบัน ปรับง่าย เร็ว เห็นผลลัพธ์เป็นส่วน ๆ แต่ต้อง migrate code ให้ดีกว่าเดิมตอนไปใช้จริง

```python
# load data collected from team1
import pandas as pd


url = 'http://128.2.25.78:8080/private/log1.clean'
df = pd.read_csv(url)
df.head()
```

	dayIdx	user	userAvgTime	location	dow	isWeekend	time
0	0	Pittsburgh66Correy	7.045001	Pittsburgh	6	True	0.000000
1	1	Pittsburgh66Correy	7.045001	Pittsburgh	7	True	6.883333
2	2	Pittsburgh66Correy	7.045001	Pittsburgh	1	False	6.816667
3	3	Pittsburgh66Correy	7.045001	Pittsburgh	2	False	7.383333
4	4	Pittsburgh66Correy	7.045001	Pittsburgh	3	False	0.000000

Data was preprocessed externally, identifying the time at a given day when the light was first turned on
12pm). Weather and sunrise information is not included here, though that'd be important. If the light w
this morning (quite common), 0 is recorded.

```python
[ ] # just data encoding and splitting X and Y

    X = df.drop(['time'], axis=1)
    YnonZero = df['time'] > 0
    Y = df['time']

    from sklearn import preprocessing
    # leDate = preprocessing.LabelEncoder()
    # leDate.fit(X['date'])
    # leDate.transform(X['date'])

    X=X.apply(preprocessing.LabelEncoder().fit_transform)
    X
```

An example of a notebook interleaving code and text cells and showing the output of a code cell below the cell. [Online-only figure.]

Jupyter Notebook since 1988

- A notebook consists of a sequence of cells, where code in cells can be executed one cell at a time, and the output of a cell's execution is shown below the cell.
- Computational notebooks have been available since 1988 with Wolfram Mathematica,

The Software-Engineering Process: An Engineering Mindset

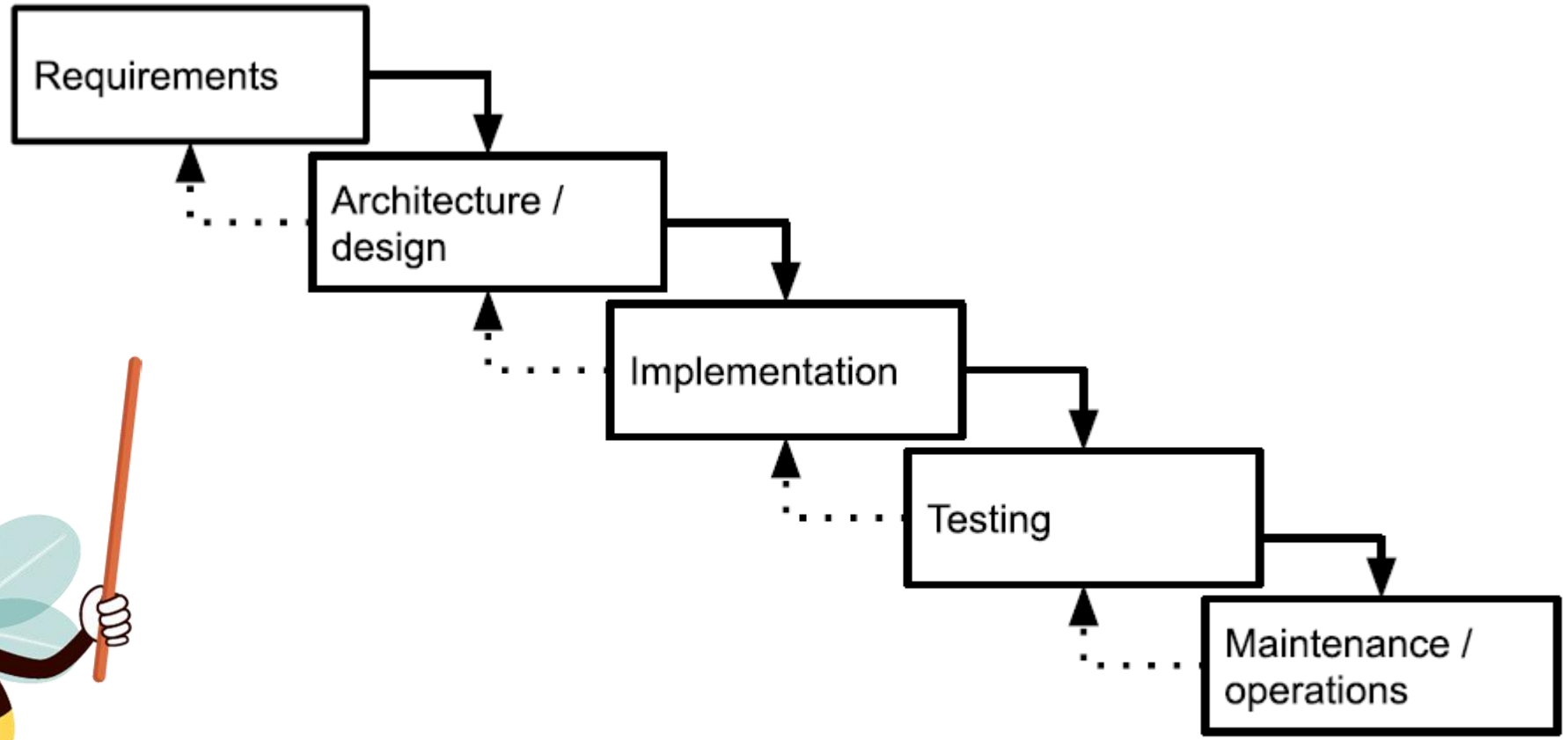
- **Defining the Lifecycle:** Focuses on requirements gathering, design, implementation, testing, deployment, and maintenance
- **Planning Over Prototyping:** Experience shows that jumping straight into code is dangerous; early planning avoids expensive corrections later
- **Process Strictness:** Relies on non-coding activities like version control, issue tracking, and progress monitoring to ensure delivery is on time and within budget

SE Process เน้นกระบวนการ non-coding ก่อน coding

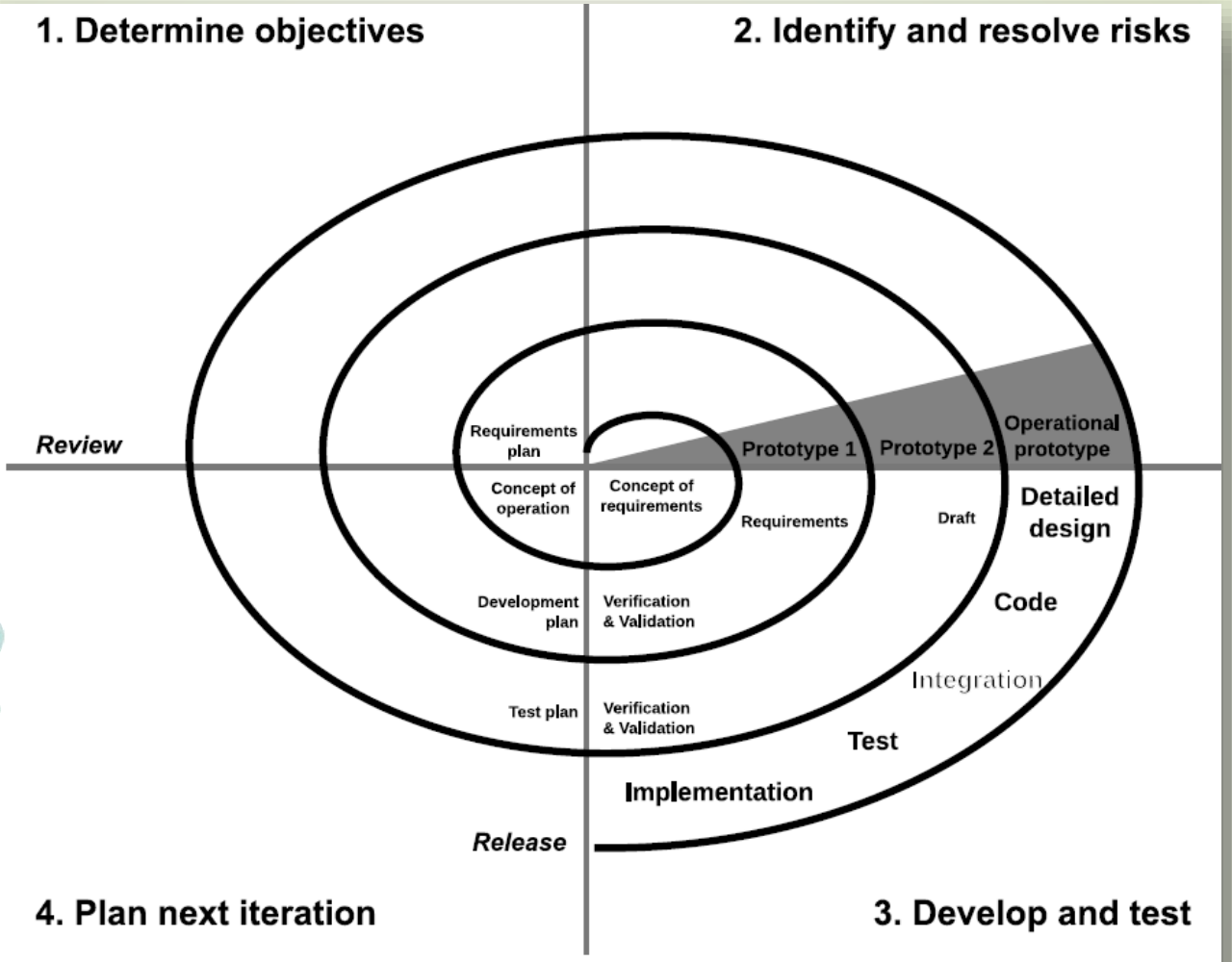
Software Process Models: From Waterfall to Agile

- **Waterfall Model:** A sequential approach emphasizing that requirements and design should be established before coding begins
- **Spiral Model:** Focuses on risk reduction by building a series of increasingly complete prototypes, starting with the most uncertain parts
- **Agile Methods:** The most common modern approach; emphasizes frequent iteration (Sprints), continuous replanning, and reacting to changing user needs
- **Trade-offs:** High-risk/stable systems (aerospace) favor Waterfall/Spiral; rapidly evolving products (social media) favor Agile

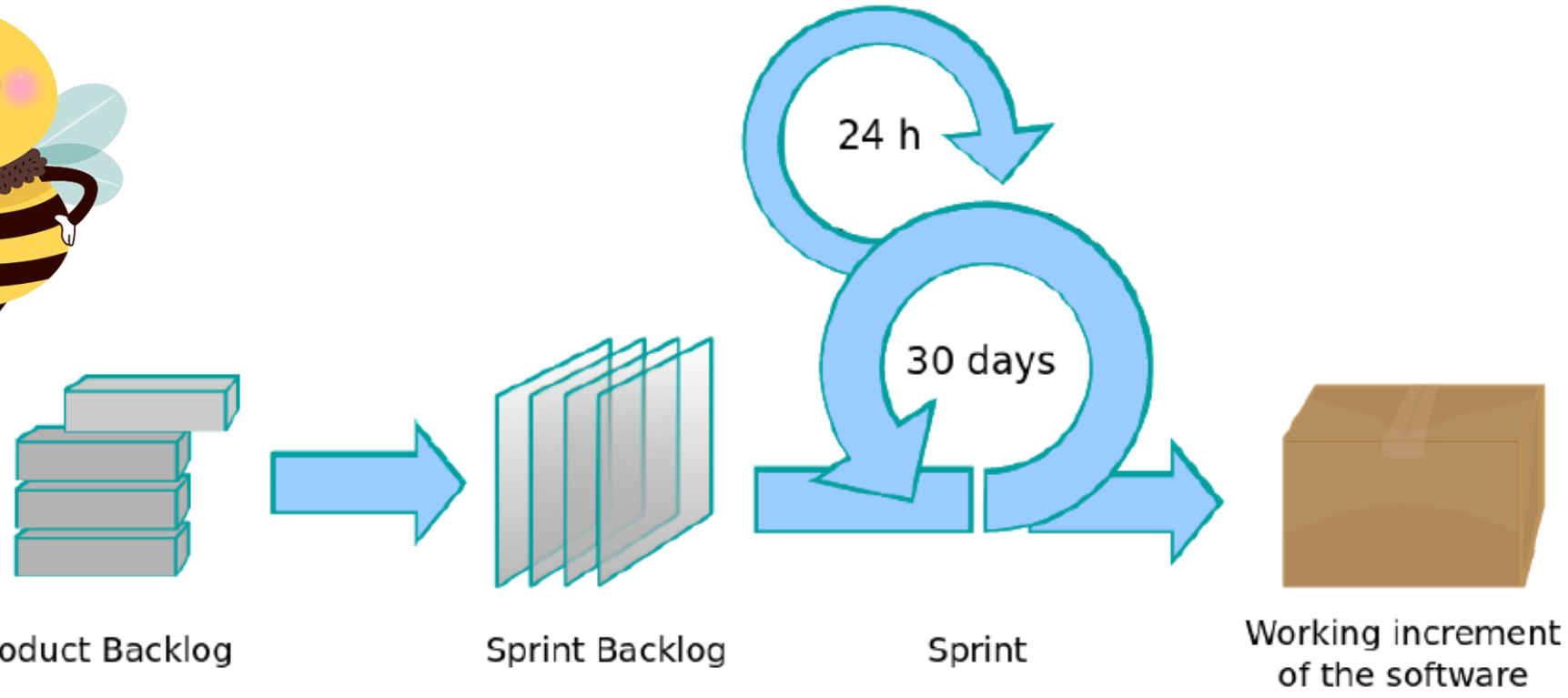
SE Process มีหลายแบบ เลือกให้เหมาะสม



A simple visual representation of the waterfall process model.



The spiral model suggests building a series of prototypes, interleaving requirements, design, implementation, and testing activities, starting with the most risky components. Figure based on Boehm, Barry W. "A Spiral Model of Software Development and Enhancement." Computer 21, no. 5 (1988): 61–72.



Agile development with constant iteration, for example, through sprints and weekly standup meetings. (CC BY-SA 4.0, Lakeworks) [Online-only figure.]

Tensions Between Data Science and Engineering

- **Science vs. Engineering:** Traditional software projects are generally more predictable and easier to decompose into parallel tasks than ML model development
- **Conflicts in Practices:** Software engineers often criticize the lack of abstraction, testing, and documentation in exploratory data science code (the "notebook" style)
- **Estimation Challenges:** Data scientists often cannot guarantee if or when a specific accuracy level is achievable, which frustrates project managers used to clear milestones
- **Misunderstanding Work:** Engineers may have unrealistic expectations (e.g., "zero false positives"), while data scientists may resent doing "infrastructure work"

Scientist vs Engineer ทำงานแตกต่างกัน อาจวิจารณ์วิธีการหรือผลงานของฝ่ายตรงข้ามได้

Idealized Integrated Process

- **System-Wide Perspective:** Successful integration requires a process that encompasses both ML and non-ML components
- **Decomposition:** Start with overall system requirements to derive specific requirements for individual components
- **Independent Development:** Allow components to be developed with their own best-fit processes (e.g., Agile for UI, CRISP-DM for models)
- **Continuous Integration:** Ensure that ML and non-ML parts are integrated and tested together early and often

Integrated Process ต้องมีลักษณะทั้ง 4 ประการตามที่ระบุไว้ในที่นี้ คือ system-wide, decomposition, Independent Dev, CI.

Integrated Process for ML-Enabled Systems

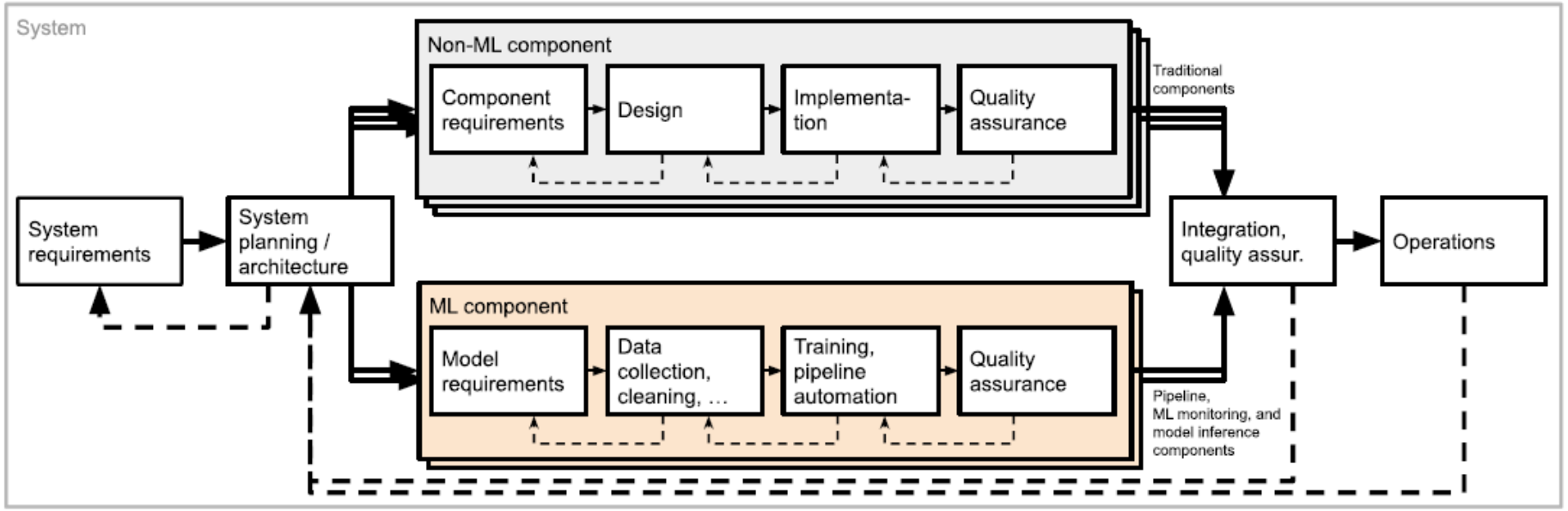
ลักษณะทั้ง 4 ประการ

-system-wide,

-decomposition,

-independent Dev,

-CI.



An iterative development process for a system, integrating distinct iterative development processes of the ML and non-ML components of the system.

Model-First vs. System-First Development

- **Model-First Approach:**
  - **Focus:** Develop the model to establish feasibility before building the system around it
  - **Pros:** Avoids heavy engineering investment if the core ML task is impossible
  - **Cons:** System design and user needs can become an afterthought
- **System-First Approach:**
  - **Focus:** Start with a clear product vision where ML is a planned component (e.g., adding a feature to existing software)
  - **Pros:** Fosters ส่งเสริม system-wide planning and better user experiences
  - **Cons:** Risk of setting unrealistic goals for data scientists based on promises to clients

Approach จะเลือกแบบไหน ต้องดูที่ระบบนั้นจุดเด่นคือ ML หรือไม่ เช่น ระบบ Audio Transcription (ต้องเลือก model-first) หรือ ระบบตรวจจับ Fraud ซึ่งมีวิธีมาตรฐานอยู่แล้ว (เป็น system-first)

Summary and Key Takeaways

- **No Single "Best Practice":** The choice between model-first and system-first depends on the project's core risks and maturity
- **T-Shaped Professionals:** Success requires team members who have deep expertise in their field but broad enough knowledge to understand and respect the processes of others (Unicorn จะหายาก คือผู้ที่รู้ดีทั้งสองเรื่อง)
- **Iterative Planning:** Both fields recognize that change is constant; the goal is to integrate these iterations into a cohesive, reliable software product

เลือกอย่างไรให้เหมาะสม ต้องใช้ T-shaped team และทั้ง ML and non-ML components ต้องใช้วิธี iterative process อยู่แล้ว ต้องให้มารวมกันให้ได้ เช่น ใช้ DevOps และ MLOps พร้อมกัน

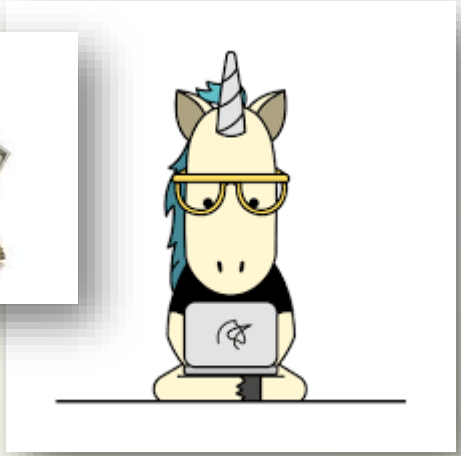
2. Interdisciplinary Teams

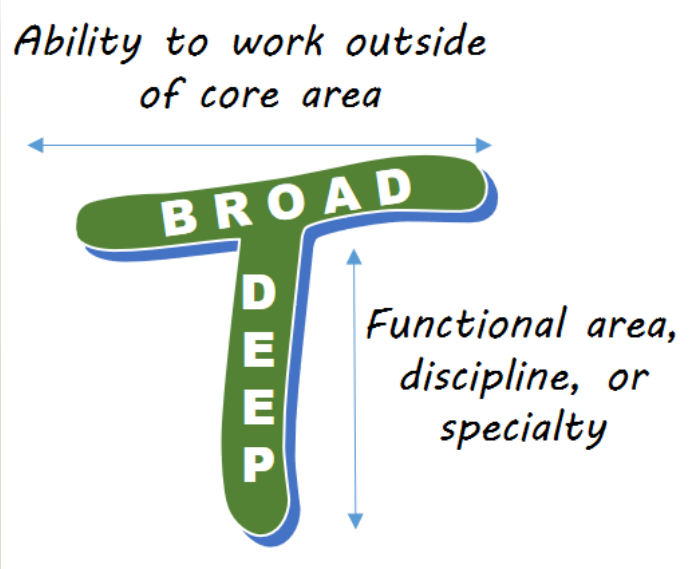
Interdisciplinary Team

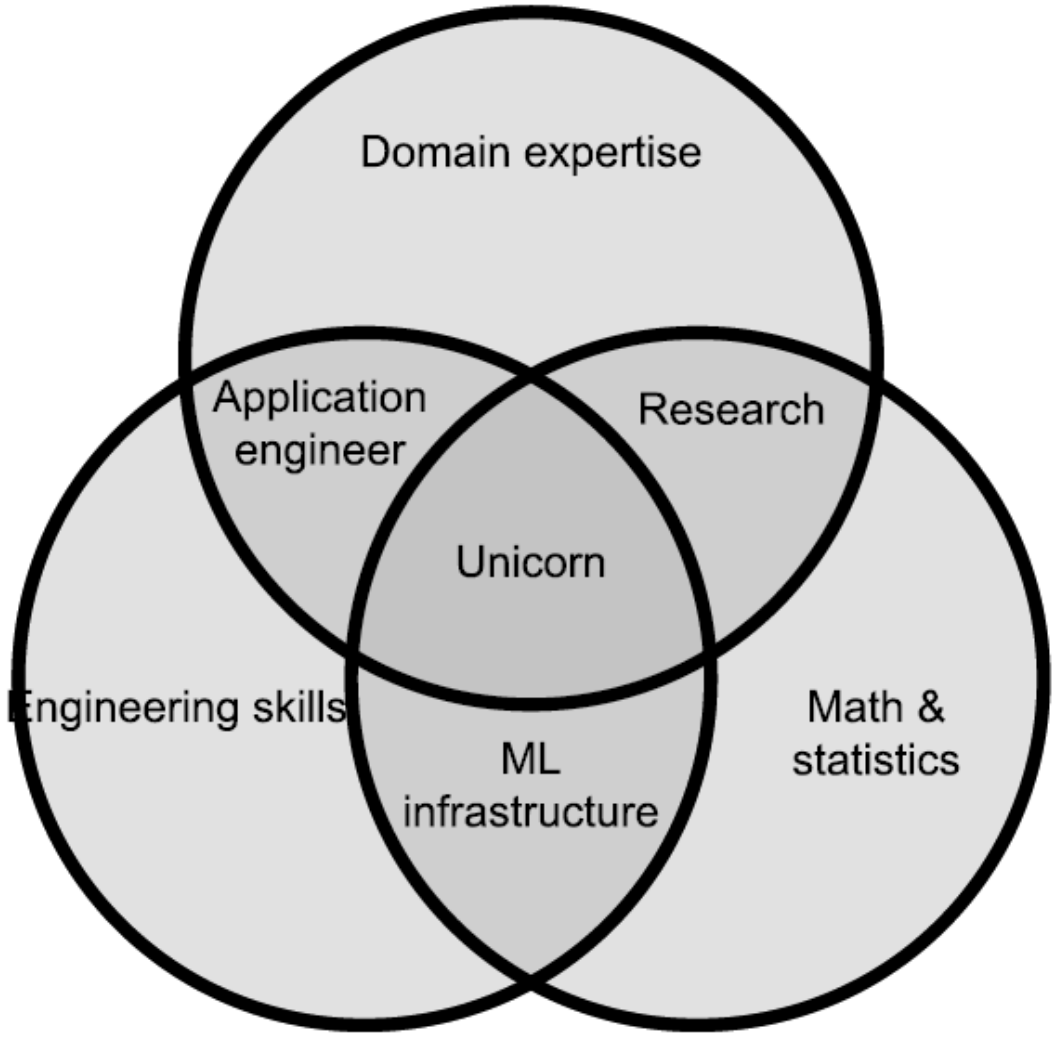
- **The Necessity of Teams:** Building ML-enabled products is too complex for a single individual; it requires a **division of labor and expertise**
- **Core Challenge:** Getting professionals with widely different backgrounds (data science, software engineering, operations, legal) to work toward **joint goals**

The Myth of the "Unicorn" Developer

- **Definition:** A "unicorn" is a rare developer who possesses deep expertise in statistics, software engineering, business, and a specific domain (e.g., medicine)
- **The Reality:** Such individuals are rare and expensive; relying on them does not scale for nontrivial systems
- **Team Strategy:** Instead of searching for unicorns, organizations should assemble **interdisciplinary teams** where members contribute distinct, complementary skills







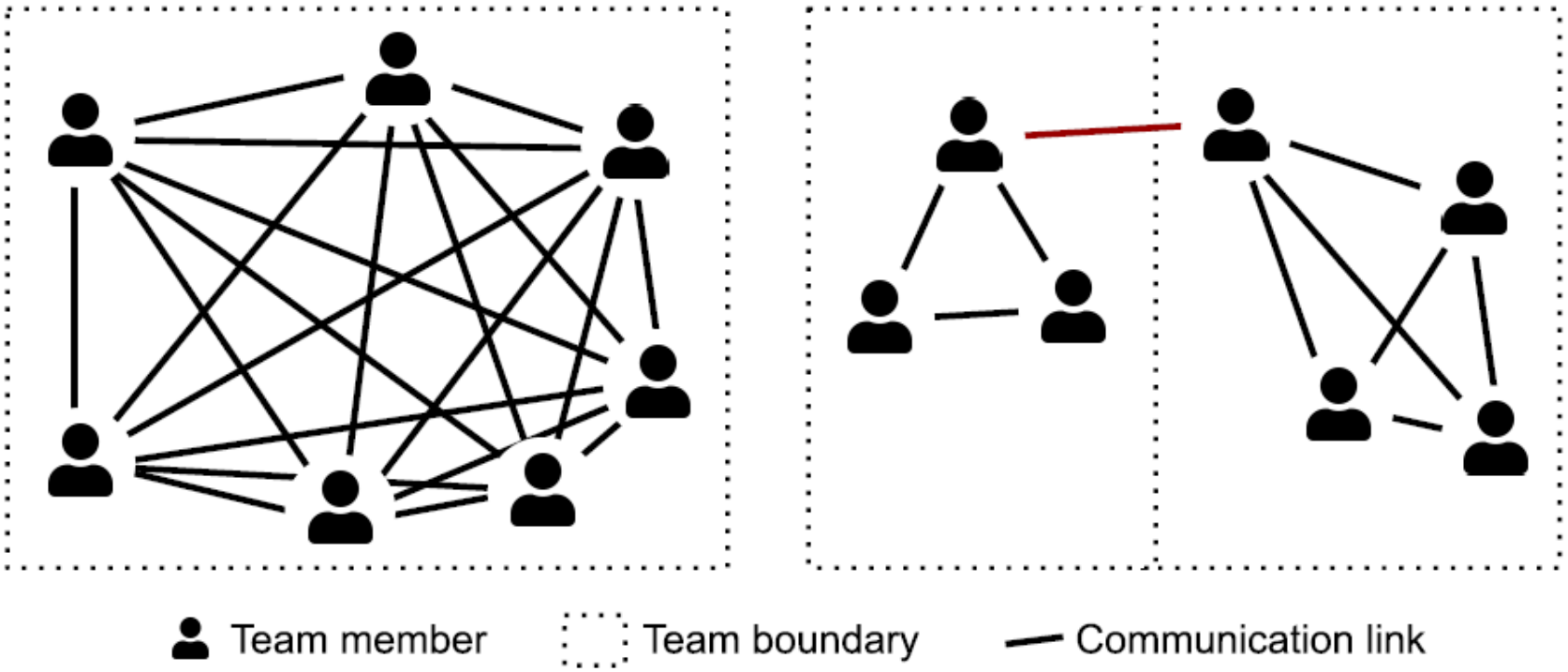
Unicorns at the intersection of various skills, a variant of common visualizations originally by Steve Geringer.

- UNICORN are not enough
  - หายาก
  - มี cost ส่วนอื่น อีกมาก

Coordination Costs and Information Hiding

- **Communication Overhead:** As team size grows, communication links increase quadratically, making large, unstructured teams inefficient
- **Information Hiding:** A strategy to limit coordination by using **stable interfaces**. Teams only need to know "what" a component does (the API), not "how" it works internally
- **The ML Friction Point:** In ML projects, ML component interfaces are often **unstable and difficult to document** (e.g., describing data distributions or model limitations), leading to frequent cross-team friction

แนวโน้มคือ จัดการ link ของการสื่อสารไม่ให้มาก ซ่อนรายละเอียดเปิดเผยเฉพาะ API แต่โชคร้าย คือ ML component API มักจะ unstable ในตอนแรกและอาจเปลี่ยนบ่อย

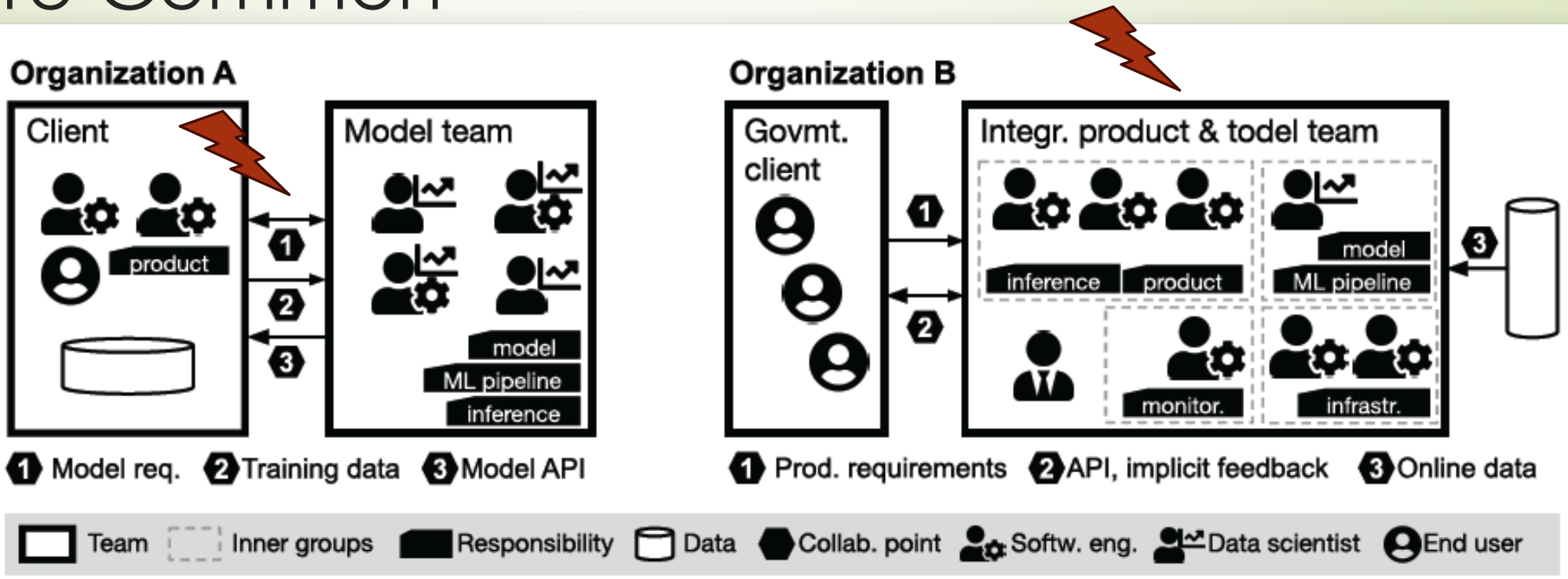


A group with n members has n(n-1)/2 communication links (left), but dividing people into multiple teams and coordinating between the teams through representatives for each team can significantly reduce the number of communication links, here from 21 to 10 for the same number of people.

Comm. Link ลดลง ทำให้เกิดการแยกทีมชัดเจน ส่งผลว่า ถ้า component ต้องทำด้วยบางคนในสองทีมนี้ จะเกิดปัญหาตาม Conway's law ที่ว่าโครงสร้างทางเทคนิคเป็นภาพสะท้อนของโครงสร้างองค์กรที่สร้างมัน

Conflicts Within and Between Teams are Common

มักเกิด conflict ระหว่างทีม



Internal team structures and collaboration points of two organizations building products with machine-learning components. Conflicts often arise at these collaboration points.

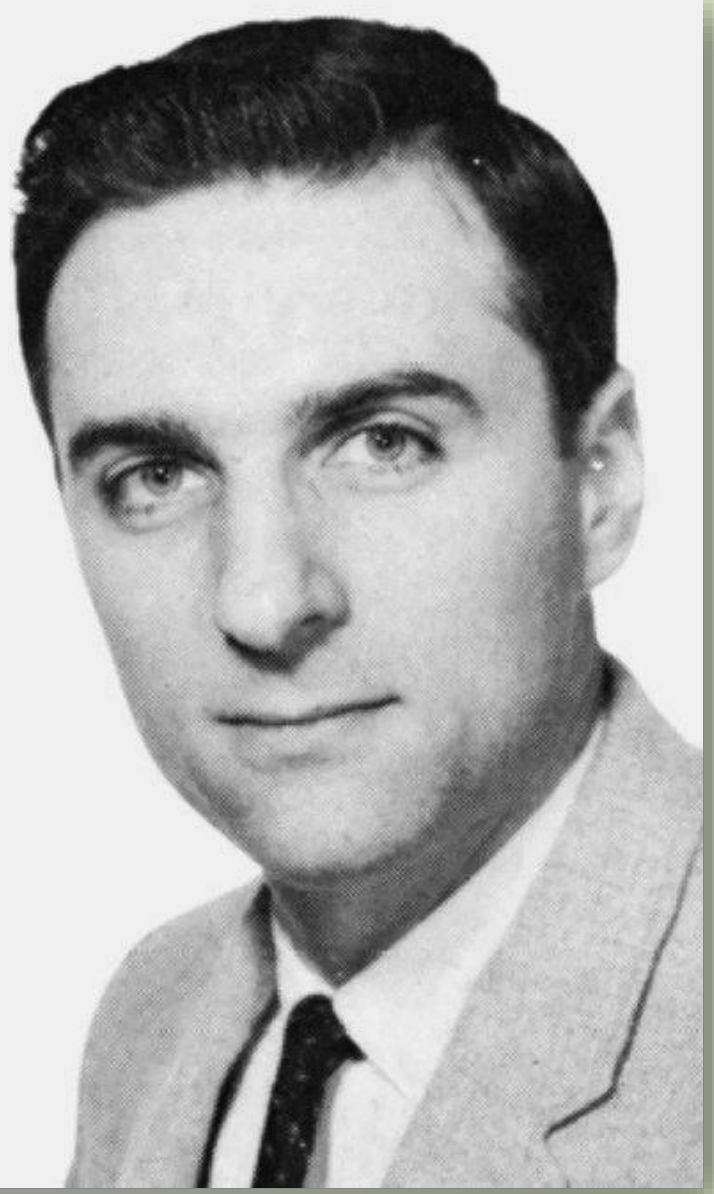
Socio-Technical Congruence (Conway's Law)

- **Conway's Law:** The observation that a system's technical structure will mirror the communication structure of the organization that built it
- **Congruence ความสอดคล้องกัน:** High congruence occurs when team boundaries align with module boundaries. Mismatches (e.g., two teams sharing one module) lead to slower development and more defects

Conway's law บอกว่า โครงสร้างของทีมในองค์กร ส่งผลกับการสื่อสารมาก และส่งผลกับงานที่ทำได้ ถ้าไม่สอดคล้องกัน ทีมเดียวกันทำ component เดียวกันจะสอดคล้อง **(ถ้าคนละทีมแต่ทำ component เดียวกันจะไม่สอดคล้อง)** จะส่งผลให้เกิดความล่าช้า และ defect ได้

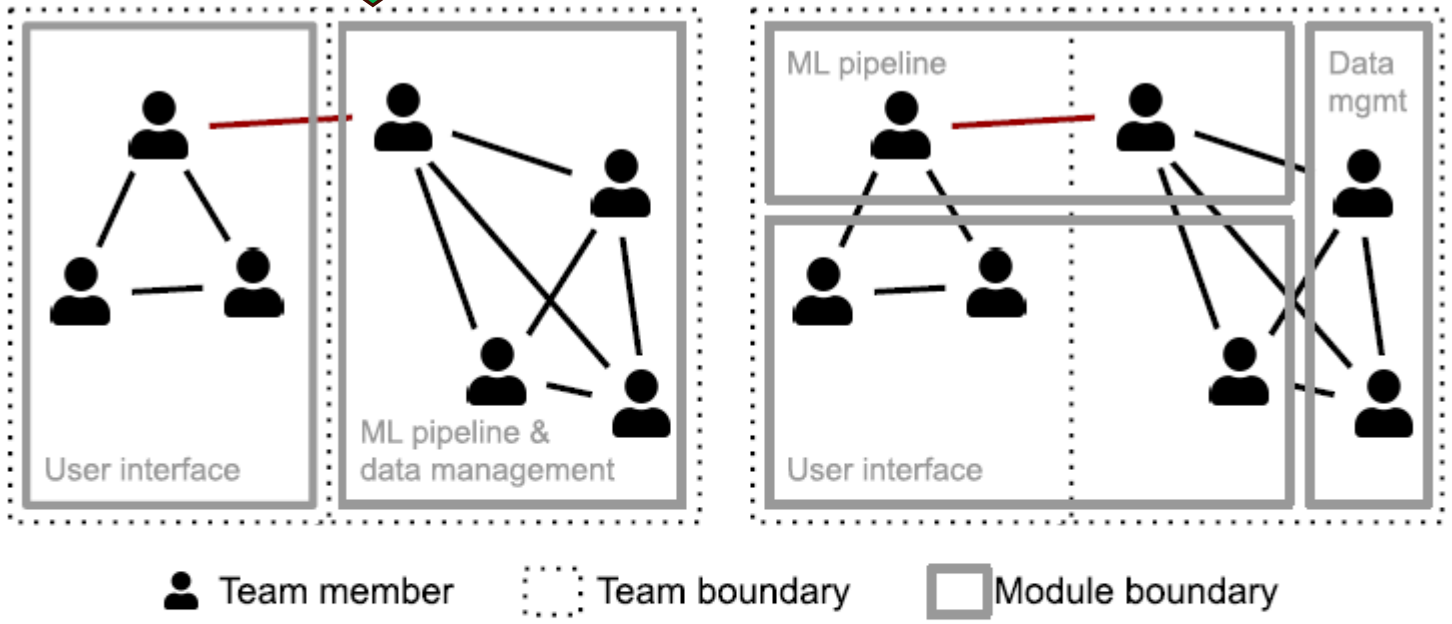
Conway's Law

Any organization that designs a system will produce a design whose structure is a copy of the organization's internal communication structure.



Conway's Law

Conway's law ที่ว่าโครงสร้างทางเทคนิคเป็นภาพสะท้อนของโครงสร้างองค์กรที่สร้างมัน



High congruence (left) where team and module boundaries align versus very poor congruence (right) where members of two teams are responsible for three modules and two modules are maintained by members of different teams, sometimes without direct communication links.

การแบ่งงาน (ดูแล component/module) ควรสอดคล้องกับโครงสร้างของทีมด้วย

Challenges to "Divide and Conquer" in ML

อย่าแบ่งฝ่าย

- Divide and Conquer is not always easy to apply
- **ML model Interfaces are rarely stable:** Information hiding relies on interfaces being *stable*. Ideally, they are designed once at the beginning and never changed. ML Interfaces can rarely be fully designed up front (exploratory and iterative nature of model)
- **Non-Local Qualities:** Critical attributes like **safety, fairness, and security** cannot be localized to a single module; they emerge from the interaction of all components
- **Diffusion of Responsibility:** Compartmentalization can lead teams to assume "someone else" is handling system-wide issues like bias or operational efficiency
- **Expertise Overlap:** Engineers need ML literacy to plan for model mistakes, and data scientists need engineering literacy to build production-ready pipelines

Divide and Conquer ดี แต่ไม่ง่ายเสมอไป เพราะการแยกขาดกันต้องผ่าน interface และ ML interface บอกชัดเจนไม่ได้และอาจเปลี่ยนได้ เพราะ? Quality ที่ได้เกิดจากทุก component ทำงานร่วมกัน ได้มาจากความรับผิดชอบร่วมกันของทุกคนในทีม การจัดทีมจึงมีความสำคัญ อย่าแบ่งฝ่าย

Awareness as a Coordination Strategy

- **Broadcast vs. Hide:** While information hiding reduces noise, **awareness** involves actively broadcasting changes to those who might be affected
- **Targeted Awareness:** Developers should subscribe to relevant updates (e.g., Slack alerts for model retraining) to avoid information overload while staying informed of changes in data quality or latency
- **Formalizing Links:** Documenting "who to contact" for specific data or model issues is essential, especially when personal relationships are lost due to turnover

Information hiding ช่วยลด overload link ก็จริง แต่ต้องให้มี awareness เกิดขึ้นโดย broadcasting changes ไป target dev ที่ต้องรู้ด้วย ต้องมี formal link ของผู้ที่ต้องติดต่อ เผื่อกรณีคนสนิทต้องออกจากทีม

Conflicting Goals and T-Shaped People

- **The Conflict:** Data scientists may optimize for **accuracy**, while operators prioritize **low latency**, and lawyers focus on **privacy compliance**
- **T-Shaped Professionals:** Individuals with **deep expertise in one area** (the vertical bar) and **broad knowledge of others** (the horizontal bar)
- **Mutual Appreciation:** T-shaped members can negotiate trade-offs effectively (e.g., a data scientist accepting lower accuracy to fulfill an explainability requirement needed by the UI team)

การมี Interdisciplinary team อาจเห็นการยึดติดกับ goals แต่ละสาขาวิชาชีพต่างกัน และเกิด conflict goals ในที่สุด ต้องใช้ T-Shaped team เพื่อจะมีความรู้และ trade-offs conflict goals ได้ดี

Team Dysfunctions: Groupthink

- **Groupthink** *is* the tendency of a group to minimize conflict by (1) not exploring alternatives, (2) suppressing dissenting คัดค้าน views, and (3) isolating the group from outside influences
  - Groupthink leads to irrational decision-making where the group agrees with the first suggestion or a suggestion of a high-status group member
  - Symptoms include teams that overestimate their ability and closedmindedness in general, with pressure toward uniformity and self-censoring
- **ML Context:** Groupthink can lead teams to ignore safety/fairness risks in order to meet a deadline or to adopt "hype" technologies without exploring better alternatives
- **Interventions:** Teams can combat this by **rewarding dissent**, assigning a "devil's advocate," and involving diverse stakeholders early in the requirements phase

ถึงแม้ว่าเราจะเชื่อ Conway's law แต่ต้องระวังเรื่อง Groupthink ไม่ดี? Groupthink มักเกิดในทีมที่มี**ความเหมือนกันมากเกินไป (Homogeneity)** เช่น มีพื้นฐานการศึกษาหรือประสบการณ์เหมือนๆ กัน และอยู่ในวัฒนธรรมองค์กรที่ปิดกั้นการรับข้อมูลจากภายนอก

Groupthink is a phenomenon where people tend to confirm with group decisions to avoid feeling outcast, leading to errors in decision making.

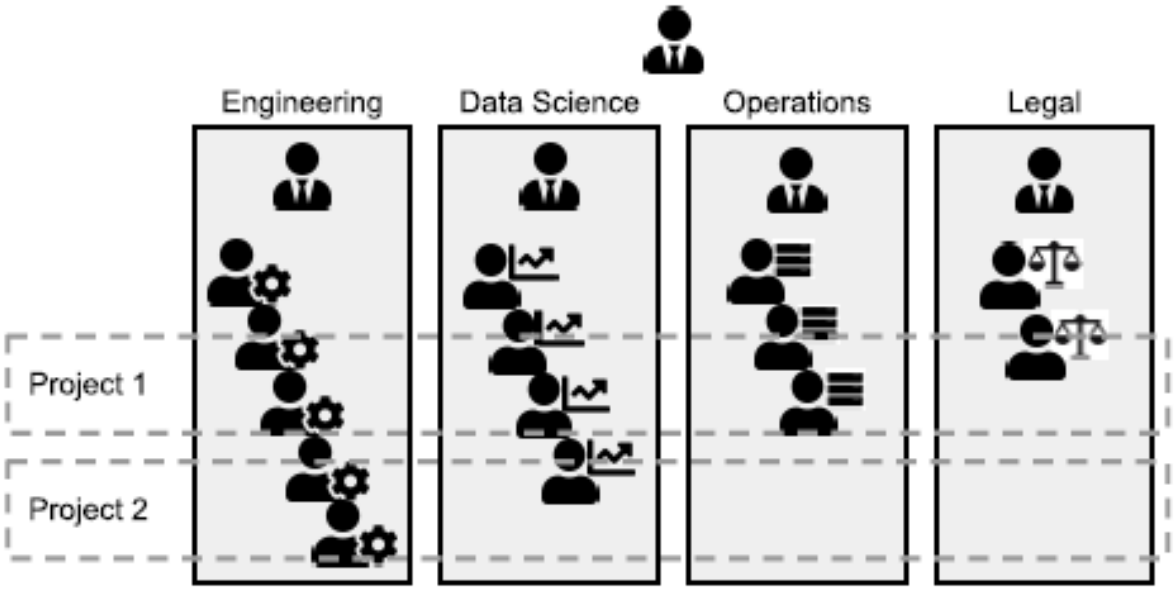
ไม่ให้เกิด Groupthink สมควรให้มี stakeholder ที่หลากหลาย มี ผู้ที่เห็นต่างมาร่วมด้วย

How to Avoid Groupthink for Team Success (+Examples)

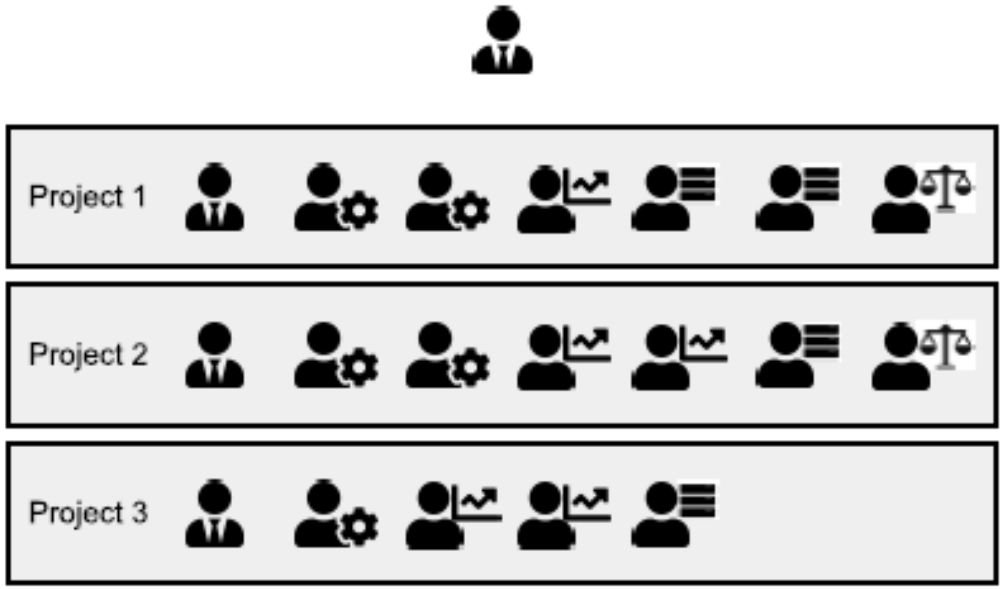
Team Structure: Matrix vs. Project Organizations

- **Matrix Organization:** Experts stay in functional departments (e.g., "The Data Science Dept") and are "lent" to projects. **Pros:** Strong peer learning. **Cons:** Conflicting loyalties between dept and project
- **Project Organization:** Members belong exclusively to one project team. **Pros:** High focus and agility. **Cons:** Potential for reinventing wheels and isolation from field-specific innovations
- **Hybrid Models:** Using "Chapters" or "Guilds" to foster cross-team learning while maintaining project-focused teams

Groupthink ไม่ดี? Groupthink มักเกิดในทีมที่มี**ความเหมือนกันมากเกินไป (Homogeneity)** เช่น มีพื้นฐานการศึกษาหรือประสบการณ์เหมือนๆ กัน และอยู่ในวัฒนธรรมองค์กรที่ปิดกั้นการรับข้อมูลจากภายนอก



(a) Matrix organization



(b) Project organization

In a matrix organization, people are grouped into departments that then lend them to specific projects. In a pure project organization, team members belong exclusively to a team.

Chapters and Guilds

Chapters และ Guilds เป็นรูปแบบของเครือข่ายการสื่อสารภายในองค์กรที่นำมาใช้เพื่อแก้ปัญหาความสมดุลระหว่างการทำงานแบบเน้นโครงการ (Project-based) และการพัฒนาความเชี่ยวชาญเฉพาะทาง (Functional expertise)

1. **Chapters (หน่วยเฉพาะทางในสายงาน)**

• **นิยาม:** เป็นกลุ่มของพนักงานที่มีทักษะหรือบทบาทหน้าที่เหมือนกัน (เช่น กลุ่ม Data Scientists หรือกลุ่ม UI Designers) แต่ถูกแยกย้ายไปทำงานอยู่ในทีมโครงการ (Squads) ต่างๆ

• **หน้าที่หลัก:** มุ่งเน้นไปที่การกำหนดมาตรฐานในสายงาน (Field-specific standards) และการแลกเปลี่ยนความรู้ในเชิงลึกระหว่างผู้เชี่ยวชาญในด้านเดียวกัน เพื่อไม่ให้เกิดภาวะ "ต่างคนต่างทำ" จนต้องเสียเวลาสร้างสิ่งที่คนอื่นในบริษัทเคยทำสำเร็จไปแล้ว (Reinventing wheels)

• **ความสำคัญ:** ช่วยให้ผู้เชี่ยวชาญที่อาจจะรู้สึกโดดเดี่ยวในทีมโครงการ (เช่น มี Data Scientist เพียงคนเดียวในทีมวิศวกร) ยังมีพื้นที่ในการเรียนรู้จากเพื่อนร่วมอาชีพในองค์กรเดียวกัน

2. **Guilds (กลุ่มความสนใจร่วมกัน)**

• **นิยาม:** เป็นชุมชนที่เปิดกว้างมากกว่า Chapter โดยเป็นการรวมตัวของคนจากหลากทีมและหลากสาขาอาชีพที่มีความสนใจใน **"หัวข้อพิเศษ"** ร่วมกัน

• **หน้าที่หลัก:** เป็นพื้นที่สำหรับการพูดคุยและแลกเปลี่ยนไอเดียในหัวข้อที่ตัดสลับข้ามสายงาน (Cross-cutting topics) เช่น Guild ด้านความปลอดภัย (Security), Guild ด้านความยุติธรรมของ AI (Fairness), หรือ Guild ด้านกฎหมายความเป็นส่วนตัว

• **ประโยชน์:** ช่วยสร้างวัฒนธรรมการทำงานร่วมกันและสร้างความตระหนักรู้ในประเด็นสำคัญที่อาจถูกละเลยไปหากโฟกัสเพียงแค่การส่งมอบฟีเจอร์ของโครงการตนเอง

Learning from DevOps and MLOps Culture

- **Breaking the "Us-vs-Them" Mentality:** Traditionally, developers wanted features while operators wanted stability. DevOps/MLOps aligns them through **joint goals**
- **Joint Tools & Vocabulary:** Shared artifacts like **containers (Docker)** and **versioning infrastructure** force different roles to speak the same technical language
- **The Vision (DevML):** A culture where software engineers and data scientists collaboratively handle requirements, risk analysis, and integration testing from day one

การมองแบบ system-wide view และ Groupthink ทำให้เกิดการใช้ DevOps (Developer+Operation) และ MLOps (DS+Operation) ซึ่งดีแนวโน้มจะมี แนวคิดในอนาคตแบบ DevML แนวการทำงานร่วมระหว่าง SE + DS การใช้ Joint tools and Vocab ช่วยให้ทุกกลุ่มต้องรู้และคุยภาษาเทคนิคเดียวกัน

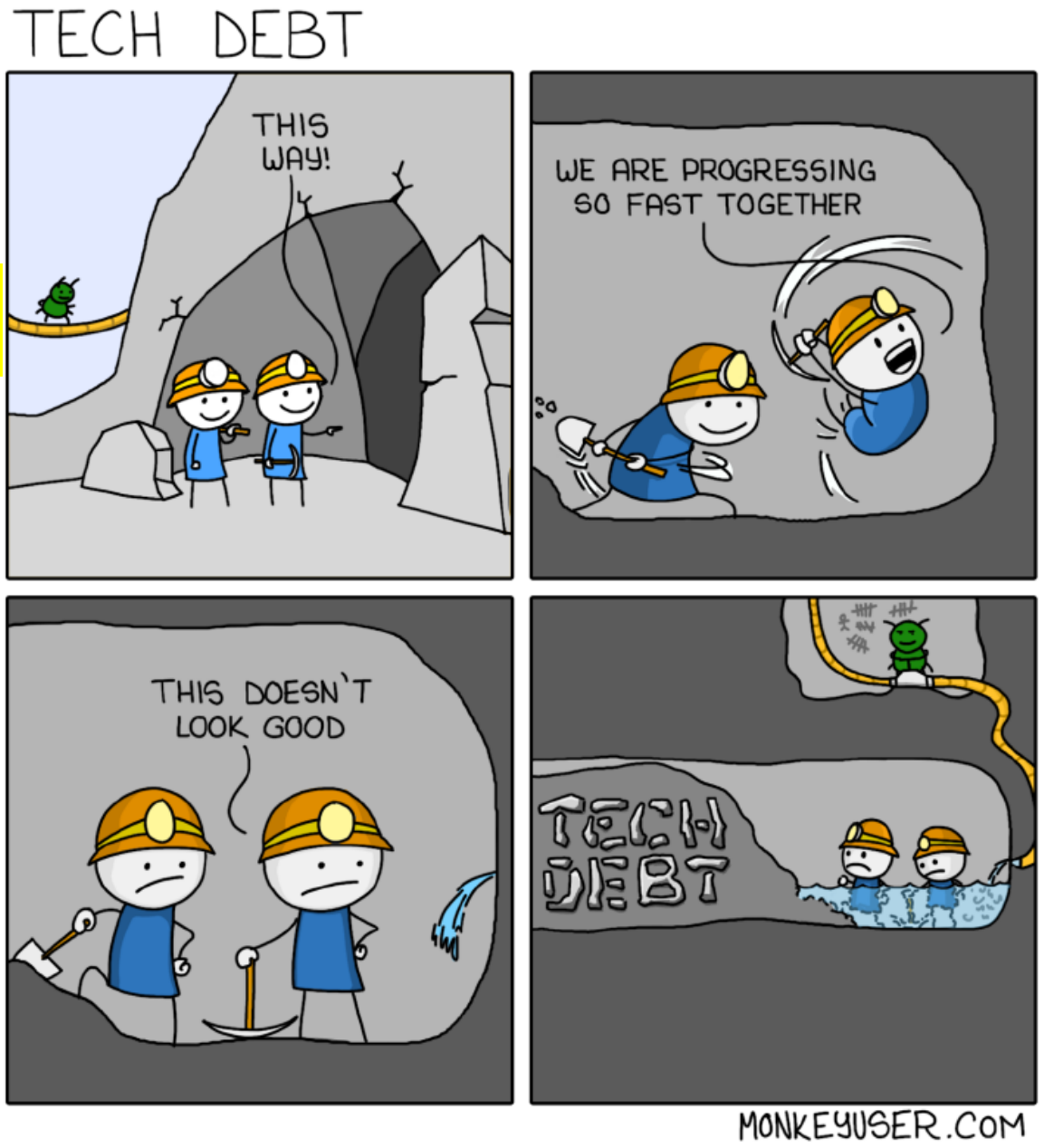
3. Technical Debt

Technical Debt in ML Systems

- **The Metaphor:** Technical debt is a trade-off between short-term benefits (e.g., faster release) and long-term costs (e.g., lower productivity, rework)
- **The "Interest":** Expedient วิธีสะดวก design choices today set up a technical context that makes future changes more costly or impossible
- **Communication Tool:** for developers to explain why time must be spent on infrastructure and maintenance rather than just new features
- **The Danger:** Accumulated debt can eventually "suffocate" ขัดขวาง a project, making it impossible to continue development without first fixing old decisions

Technical debt เกิดขึ้นง่ายมากเป็นพิเศษใน MLS Development

ตัดสินใจทำอะไรง่าย ๆ เร็ว ๆ มักจะเกิด debt ที่ทิ้งไว้ให้จ่ายเสมอ



Technical debt comic by Cornel [Online-only figure.]

Deliberate vs. Inadvertent Debt

- **Deliberate** จงใจ **Debt:** A conscious decision to take a shortcut to meet a deadline, knowing it must be fixed later
- *Example:* Manually copying a model to a robot instead of building an automated pipeline to get a prototype out faster (ติดตั้งด้วยมือ โดยไม่ใช้ automated pipeline เร็วแต่เกิด debt)
- **Inadvertent** ไม่ตั้งใจ **Debt:** Incurring costs without realizing it, often due to a lack of experience or understanding of engineering best practices
- *The Trap:* Developers may only realize their past decisions were inefficient once they are already being slowed down by them (ด้วยความไม่รู้ ขาดประสบการณ์ มักเรียนรู้ได้ ตอนพลาดไปแล้วครั้งแรก)

Deliberate จงใจ ตั้งใจ debt มักเกิดเสมอ แต่มีบางส่วนเป็นแบบ

Inadvertent debt ไม่ตั้งใจ เพราะไม่รู้ขาดประสบการณ์

Prudent vs. Reckless Debt

- **Prudent Debt:** A strategic investment where the short-term benefit (e.g., acquiring a user base in a competitive market) outweighs the long-term maintenance costs
- **Reckless Debt:** Taking massive shortcuts due to immense stress or ignorance, leading to costs that may bankrupt the project's future
- **The Goal:** Ideally, all debt should be **deliberate and prudent**, where the team understands the trade-offs they are making

Prudent = รอบคอบ Reckless = ประมาท

การเกิด Tech Debt ควรเป็น Deliberate and Prudent เสมอ

Why ML Projects Accumulate Debt Easily

- **Unnecessary ML:** Using machine learning as a "gimmick" or to solve a problem where simple heuristics would suffice creates massive, unneeded maintenance debt
- **Process Neglect:** Skipping "Responsible Engineering" practices (like threat modeling or fairness audits) creates a hidden backlog of risks and rework

การเลือกทำ ML component โดยไม่จำเป็น ทำให้เกิด debt ได้ ดังนั้นอย่าใช้ ML ถ้าไม่คุ้มหรือได้ผลดีกว่า และ ML component มักจะถูกละเลยเรื่อง responsible eng. เสมอ

ML-Specific Debt: Data and Infrastructure

- **Data Debt:** Dependencies on external data sources that may change quality, quantity, or distribution over time without warning
  - *Risk:* Without data schemas and drift monitoring, these issues go unnoticed until the model fails in production.
- **Infrastructure Debt:** Relying on manual steps for model training and deployment
  - *Risk:* A lack of automation makes updates tedious and error-prone, leading to "stale" models running in production

ML specific debt แบบหนึ่ง

ML-Specific Debt: Versioning and Observability

- **Versioning Debt:** Ad-hoc exploratory development in notebooks makes it difficult to reproduce results or identify what changed between releases
  - *Risk:* Inability to track down why a model's accuracy suddenly dropped due to unknown changes in training data or code
- **Observability Debt:** Delaying the investment in production monitoring tools
  - *Risk:* Developers are "blind" to production problems and must spend excessive time manually investigating robot or system failures

ML specific debt แบบหนึ่ง – ลุยทำไปเรื่อยๆ ไม่เก็บ version ในการ explore model

ML-Specific Debt: Code Quality and Testing

- **Code Quality Debt:** ML pipelines are often filled with "pipeline jungles"—glued-together scripts, dead experimental code, and duplicate feature extractions
- **Architectural and Test Debt:** Building products where multiple models consume each other's outputs without integration testing
  - *CACE Principle:* "Change Anything Changes Everything" (แก้นิดเดียว เหมือนแก้ทั้งหมด เพราะอาจโยงถึงกัน ripple effect ต้อง test ใหม่ทั้งหมด) Without testing, it is impossible to know which model in a chain is causing a system-level failure

ML specific debt แบบหนึ่ง – ไม่สนใจ quality of ML pipeline coding, ละเลยการทำ Integration testing ทุกครั้งไป

ML-Specific Debt: Requirements Debt

- **The Hidden Trap:** Skipping a rigorous requirements analysis can lead to "Requirements Debt".
- **Unintended Feedback Loops:** A failure to analyze how the system interacts with its environment can lead to models reinforcing their own biases (e.g., a delivery robot adopting risky driving because it "works" in the short term).
- **The Cost to Fix:** Requirements and design-related debt are generally much harder to "repay" later than simple code cleanups

ML specific debt แบบหนึ่ง – วิเคราะห์ req. ไม่ครบ / ไม่สนใจ feedback

Cost to fix ที่เกิดจาก req / design จะมากเกินที่จะจ่ายได้

Managing and Repaying Technical Debt

- **Repayment Strategies:**
  - **Education:** Ensuring the team knows best practices reduces inadvertent debt.
  - **Uniform Infrastructure:** Providing platforms that automate versioning and dependency tracking makes it "easy to do the right thing".
- **Tracking Debt:** Debt should be documented in issue trackers or product backlogs.
- **Repayment Events:** Adopting "fix-it" days or weeks specifically for addressing the maintenance backlog

ML specific debt แบบหนึ่ง – ถ้าต้องจ่าย debt ทำอย่างไร

Summary and Key Takeaways

- **Debt is a Tool:** Taking debt isn't inherently bad; it's a trade-off that should be made **deliberately**.
- **ML Complexity:** ML adds unique dimensions of debt (data, versioning, observability) that traditional software debt models don't fully cover.
- **The Responsible Engineer's Duty:** Recognize the "interest" ดอกเบี้ยค้างจ่าย accruing on shortcuts. If you delay design and automation, ensure you have a plan to "pay it back" before the project becomes unmaintainable

เน้น deliberate and prudent debt / ML adds new debts / วางแผนจ่ายหนี้และดอกเบี้ยถ้าเวลายิ่งนาน ดอกเบี้ยจะมาก เช่น interest = การเสียเวลาทำความเข้าใจ code ที่มี debt ยิ่งวันยิ่งซับซ้อนไปเรื่อยๆ